



LxCameraSDK (Linux)

Sample Programs Guide

Version: V2.0

Release Date: 2026.07.27

GitHub: <https://github.com/Lanxin-MRDVS/CameraSDK>

Knowledge Base: <https://hub.mrdvs.cn/>

Official Websites: <https://www.mrdvs.cn/> & <https://mrdvs.com/>

Zhejiang MRDVS Technology Co., Ltd. All Rights Reserved.



Contents

1 Overview	1
1.1 About This Document	1
1.2 SDK Contents	1
1.3 Supported Systems	1
2 First-Run Preparation	2
2.1 Install the Basic Dependencies	2
2.2 Install the SDK	2
2.3 Configure the Camera Network	3
3 C/C++ Samples	4
3.1 Build All Samples	4
3.2 Sample Functions and Usage	5
3.3 Build a Sample Separately	6
3.4 Built-in Algorithm Modes	7
4 Python Samples	7
4.1 Install the wheel	7
4.2 Run the Basic Sample	7
5 ROS1 Samples	9
5.1 Install ROS Noetic and Its Dependencies	9
5.2 Build and Start the Standard Camera Node	10
5.3 Launch Files	11
5.4 Standard Node Parameters	12
5.5 Topics and Services	12
6 ROS2 Samples	13
6.1 Install ROS2 Humble and Its Dependencies	13
6.2 Build, Start, and Verify	14
6.3 CycloneDDS	15
7 Visual Localization, Mapping, and Relocalization	16
7.1 Inputs, Outputs, and Key Configuration	16
7.2 Integration Sequence	16
8 Troubleshooting	17

1 Overview

1.1 About This Document

This document guides developers through installing LxCameraSDK, checking the camera network, and building and running the C/C++, Python, ROS1, and ROS2 samples on Linux. First-time users should complete Chapters 2 and 3 before proceeding to the sections that apply to their development environment.

***Feature support:** Data streams, the IMU, LiDAR point clouds, and built-in algorithms depend on the camera model, firmware, and algorithm license. If the program returns ‘LX_W_NOT_SUPPORT’, the current device does not support that feature; this does not indicate an SDK installation failure.

1.2 SDK Contents

Directory or File	Contents	Purpose
install.sh	SDK installation script	Installs the header files and the shared libraries for the corresponding architecture to /opt/MRDVS.
set_socket_buffer_size.sh	Network buffer script	Sets the upper limits for Linux socket send and receive buffers. install.sh invokes this script automatically.
set_firewall.sh	Firewall script	Depends on the broadcast discovery tool. If the corresponding tool is not provided, configure the rules manually.
SDK/include	C/C++ header files	Interfaces, feature enumerations, data structures, and application algorithm structures.
SDK/lib	Linux shared libraries	Provides separate versions for x86_64, 32-bit ARM, and 64-bit ARM.
Sample/C	C/C++ samples	Basic stream acquisition, callbacks, multiple cameras, IMU, LiDAR point clouds, and built-in algorithms.
Sample/python	Python wheel	Python bindings and samples included in the wheel.
Sample/ROS	ROS1 samples	Standard camera, LiDAR point cloud, visual localization, and sensor simulation nodes.
Sample/ROS2	ROS2 samples	ROS2 versions of the camera, localization, and sensor simulation nodes.

1.3 Supported Systems

Sample	Recommended Environment	Description
C/C++	Linux x86_64, arm, or aarch64	Requires gcc/g++, CMake, and OpenCV. OpenCV is used to display and save images. The shared libraries

Sample	Recommended Environment	Description
		must match the CPU architecture.
Python	Python 3	The wheel declares NumPy and OpenCV dependencies. A desktop environment is required when a sample uses a graphical interface.
ROS1	Ubuntu 20.04 + ROS Noetic	The package also retains compatibility notes for Kinetic (Ubuntu 16.04) and Melodic (Ubuntu 18.04). Noetic has reached end of official support and should be used only for existing systems.
ROS2	Ubuntu 22.04 + ROS2 Humble	Using these ROS2 samples on Ubuntu 20.04 is not recommended.

2 First-Run Preparation

2.1 Install the Basic Dependencies

On a clean Ubuntu 20.04 or 22.04 system, first install the components required by the C/C++, image-processing, and Python samples. OpenCV is not mandatory for building the C/C++ samples, but it is recommended because it enables image display and saving.

```
# Update the package index and install the C/C++ build and image-processing
dependencies
sudo apt update
sudo apt install -y build-essential cmake pkg-config libopencv-dev
# Install the Python runtime and image-processing dependencies
sudo apt install -y python3 python3-pip python3-numpy python3-opencv
```

After installation, check the system and tool versions:

```
# Check the system architecture and Ubuntu version
uname -m
lsb_release -ds
# Check the main build and runtime tools
gcc --version
g++ --version
cmake --version
pkg-config --modversion opencv4
python3 --version
```

2.2 Install the SDK

From the MRDVS directory that contains ‘install.sh’, run the installation. The script deletes the existing ‘/opt/MRDVS’ directory before reinstalling the SDK. Back up any custom files in that directory first.

```
# Install the SDK files and configure the system parameters
sudo bash install.sh
# Register the SDK shared-library path and refresh the system cache
echo /opt/MRDVS/lib | sudo tee /etc/ld.so.conf.d/mrdvs.conf
sudo ldconfig
```

Using the system shared-library cache avoids setting 'LD_LIBRARY_PATH' only for the current terminal. After installation, run the following checks:

```
# Check that the header file and main shared libraries are installed
test -f /opt/MRDVS/include/lx_camera_api.h && echo "header: OK"
test -f /opt/MRDVS/lib/libLxCameraApi.so && echo "camera library: OK"
test -f /opt/MRDVS/lib/libLxDataProcess.so && echo "data library: OK"
# Check the shared-library registration and dependencies
ldconfig -p | grep LxCameraApi
ldd /opt/MRDVS/lib/libLxCameraApi.so
```

***Architecture:** Use the 'linux_x64' libraries when 'uname -m' returns 'x86_64'; use the 'linux_arm32' libraries when the result contains 'arm'; and use the 'linux_aarch64' libraries when the result contains 'aarch64'. If the script reports 'not found matched platform', stop and confirm that the package includes shared libraries for the target architecture.

2.3 Configure the Camera Network

1. Connect the camera directly to a wired network adapter on the host, or connect both devices to the same switch. The host adapter and camera must be on the same IPv4 subnet and must not use the same IP address.
2. If the host must also access the internet, use Wi-Fi or another network adapter for the external network. If the host has only one network adapter and it is connected directly to the camera, being unable to access the internet at the same time is expected. Install online dependencies before connecting the camera.
3. For initial verification, use a dedicated wired network adapter and the following example configuration: camera IP address '192.168.100.82', host IP address '192.168.100.10', subnet mask '255.255.255.0', and no gateway. If the camera is already configured with another IP address, you do not need to change it to the address above. Set the host adapter to the camera's subnet instead. For example, if the camera IP address is '192.168.1.20', the host can use '192.168.1.10'.
4. The basic Python sample in the package connects to '192.168.100.82' by default. Using the recommended configuration above reduces the parameter changes required for the first run. If the camera uses another IP address, change the connection method as described in Section 4.2. The basic C/C++ sample connects through the device discovery results and does not require a fixed camera IP address.

5. After confirming that the network adapter is connected and the route is correct, run the sample program. Only one client can obtain control access to a camera at a time. Before running a new sample, close any other sample programs, ROS nodes, and host applications.

```
# Check the network-adapter addresses and routes
ip address
ip route
# Check the system firewall status
sudo ufw status
```

If UFW is 'active' and device discovery or streaming is blocked, open the inbound ports used by the SDK in accordance with your network security policy:

```
# Open the ports required for device discovery, control, and data transfer
sudo ufw allow 9700/udp
sudo ufw allow 9800/udp
sudo ufw allow 3956/udp
sudo ufw allow 3959/udp
sudo ufw allow 31900/udp
sudo ufw allow 32000/udp
sudo ufw allow 39560/udp
sudo ufw allow 9900/tcp
# Reload the firewall rules
sudo ufw reload
```

***Firewall script:** 'set_firewall.sh' requires 'lx_boardcast_tool' in the 'Tool' directory. If the package does not include this tool, do not rely on the script to obtain the camera IP address automatically. Use the fixed-port rules above, or ask the network administrator to restrict the source by camera IP address.

3 C/C++ Samples

3.1 Build All Samples

From the MRDVS directory that contains 'install.sh', run the following commands to build all nine samples except 'arm_local_camera':

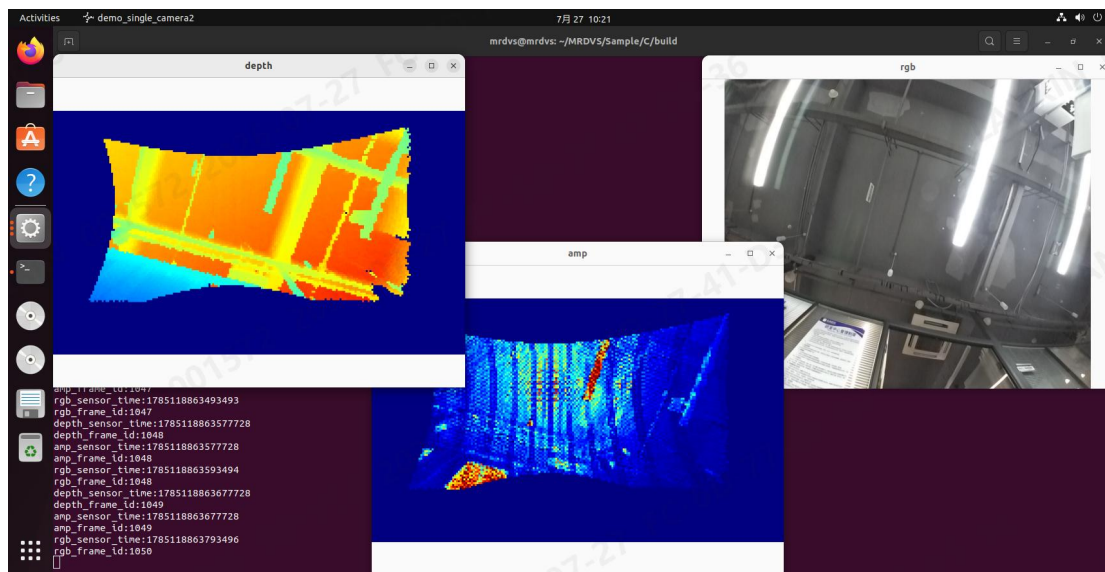
```
# Enter the C/C++ sample directory and create a separate build directory
cd Sample/C
mkdir -p build
cd build
# Configure and build all samples
cmake ..
make -j
```

```
# List the generated executables
ls bin
```

For initial verification, run the single-camera synchronous streaming sample first. The program discovers devices, opens the first camera at index 0, and prints the API version, device information, frame timestamps, and image information. Type 'q' and press Enter to exit.

```
# Run the basic single-camera streaming sample
./bin/demo_single_camera2
```

***Success criteria:** If the log contains 'DcGetDeviceList' 'success', the device IP address/serial number, and continuous depth, amplitude, or RGB frame information, the SDK installation, network, and basic streaming path are working correctly. The available data types depend on the camera's current stream configuration.



3.2 Sample Functions and Usage

Sample	Function	Usage and Exit
demo_single_camera2	Synchronously reads FrameInfo, depth, amplitude, RGB, and XYZ point clouds; can save depth.pgm, amp.pgm, rgb.png, and xyz.pcd.	Recommended for basic verification. When built with OpenCV enabled, type s to save or overwrite depth.pgm, amp.pgm, rgb.png, and xyz.pcd; type q and press Enter to exit.
demo_frame_callback	Registers a frame callback and processes FrameInfo and algorithm results in the callback.	Avoid time-consuming operations in the callback. Type q and press Enter to exit.
demo_multi_cameras	Enumerates all devices, acquires streams for each camera in an independent thread, and outputs status and frame rate.	Assess the bandwidth of the network adapter and switch. Type q and press Enter to exit.

Sample	Function	Usage and Exit
demo_imu_callback	Registers an IMU callback and outputs the timestamp, acceleration, angular velocity, and calculated frequency.	Supported only by models with an IMU. Type q and press Enter to exit.
demo_lidarpoint_callback	Obtains the XYZIRT LiDAR point-cloud structure and can save a PCD file.	Supported only by models that provide LiDAR point clouds. Type s to save and q to exit.
demo_application_obstacle_v1	Enables obstacle avoidance V1 and periodically reads the obstacle-avoidance I/O and region index.	Supported only by models with the corresponding algorithm. The program runs continuously; press Ctrl+C to stop it.
demo_application_obstacle_v2	Enables obstacle avoidance V2 and outputs the ground, point cloud, and obstacle bounding boxes.	Supported only by models with the corresponding algorithm. The program runs continuously; press Ctrl+C to stop it.
demo_application_pallet	Enables the pallet-docking algorithm and outputs x, y, and yaw.	Supported only by models with the pallet algorithm. The program runs continuously; press Ctrl+C to stop it.
demo_application_location	Demonstrates visual localization, map import/export, and external sensor-data input.	This sample is intended as a reference for custom development. Configure the map, odometry, extrinsic parameters, and algorithm parameters before running it.
demo_arm_local_camera	Calls the SDK through 127.0.0.1 on the camera's internal ARM system.	For use only in the camera's internal ARM environment. It is not included in the batch-build targets.

3.3 Build a Sample Separately

The ‘single_camera2’, ‘multi_cameras’, ‘frame_callback’, ‘application_pallet’, and ‘arm_local_camera’ directories provide standalone Makefiles. The following example uses ‘single_camera2’:

```
# From the batch-build directory, enter the single-camera sample directory
cd ../single_camera2
# Build and run the current sample
make
./demo_single_camera2
```

***application_location:** The standalone Makefile supports a basic build, but it does not enable OpenCV display. If image display is required, build the sample with the top-level CMake configuration. Before running the sample, also replace the map-import file, odometry input, and localization parameters with the actual project configuration. Do not use the sample data directly.

3.4 Built-in Algorithm Modes

Value	Enumeration	Purpose
0	MODE_ALL_OFF	Disables all built-in application algorithms.
1	MODE_AVOID_OBSTACLE	Obstacle avoidance V1.
2	MODE_PALLET_LOCATE	Pallet docking.
3	MODE_VISION_LOCATION	Visual localization.
4	MODE_AVOID_OBSTACLE2	Obstacle avoidance V2.
5	MODE_GENERIC_DATA	Custom data for the application algorithm platform.

***Device control:** When a sample exits normally, it calls ‘DcStopStream’ and ‘DcCloseDevice’. If ‘LX_E_CTRL_PERMISS_ERROR’ occurs immediately after Ctrl+C is pressed or the process exits abnormally, confirm that no process remains and wait for the camera to release control after the control heartbeat times out before restarting.

4 Python Samples

4.1 Install the wheel

From the ‘MRDVS/Sample/python’ directory, install the Python binding. If NumPy and OpenCV have already been installed through ‘apt’, use ‘--no-deps’ to avoid downloading the large packages again.

```
# Enter the Python sample directory and install the SDK binding
cd Sample/python
python3 -m pip install --user --no-deps ./lx_camera_py-1.3.3-py3-none-any.whl
# Check that the Python module can be imported
python3 -c "import LxCameraSDK; print('LxCameraSDK import: OK')"
```

4.2 Run the Basic Sample

The wheel includes the ‘single_camera’, ‘application_obstacle’, ‘application_pallet’, and ‘application_location samples’. The packaged ‘single_camera’ sample uses a specified shared-library name and camera IP address by default. To support different installation locations and camera networks, copy the sample to the current directory before the first run, then change it to use the SDK's actual installation path and the device index.

```
# Copy the installed basic sample to the current directory
python3 -c "import inspect, shutil, LxCameraSDK.Sample.single_camera as d;
shutil.copy(inspect.getsourcefile(d), 'single_camera.py')"
# Open the first camera by its index in the discovery list
```

```
sed -i 's@open_mode = LX_OPEN_MODE.OPEN_BY_IP@open_mode =  
LX_OPEN_MODE.OPEN_BY_INDEX@' single_camera.py  
# Load the shared library from the SDK's actual installation path  
sed -i  
's@main("libLxCameraApi.so")@main("/opt/MRDVS/lib/libLxCameraApi.so")@'  
single_camera.py  
# Confirm that the connection method and shared-library path were changed  
correctly  
grep -nE 'open_mode =|open_param = "0"|main\(' single_camera.py
```

After confirming that the output contains 'OPEN_BY_INDEX', 'open_param="0"', and '/opt/MRDVS/lib/ libLxCameraApi.so', run the sample:

```
python3 single_camera.py
```

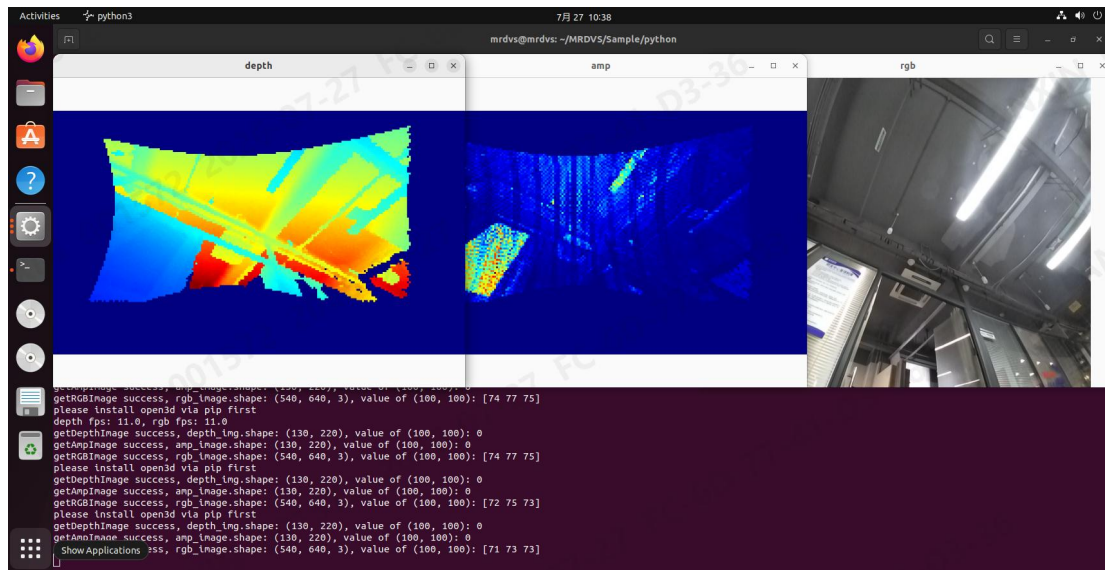
The sample registers a device-status callback, enables the depth, amplitude, and RGB streams, reads the point cloud, frame rate, and images, and displays windows through OpenCV. The program has no interactive exit key; press Ctrl+C to stop it.

***No desktop environment:** If a server or SSH terminal does not have a graphical display, the OpenCV windows fail to open. In this case, use the basic C/C++ sample first. If Python must be used, remove the window-display code from your application and retain only the stream-acquisition, saving, or data-processing logic.

***Data streams:** The basic Python sample attempts to enable the depth, amplitude, and RGB streams by default and can optionally call Open3D to display the point cloud. If the current camera model does not support one of these streams, disable the corresponding switch in the sample. Install Open3D only when a point-cloud window is required.

***Device selection:** Connecting by index is suitable for first-time verification with one camera. For multi-camera deployments, or when a specific device must be selected reliably, distinguish devices by their actual IP address, device serial number, or device ID.

***Algorithm samples:** 'application_obstacle.py' and 'application_pallet.py' contain fixed device IP addresses and algorithm configurations. They apply only to cameras that support the corresponding algorithms and must be changed to the actual device and parameters before use. 'application_location.py' is marked as incomplete and is not intended to be run directly at a customer site.



5 ROS1 Samples

5.1 Install ROS Noetic and Its Dependencies

If ROS is not installed on Ubuntu 20.04, configure the ROS repository and install Noetic first. Official support for ROS Noetic ended in May 2025. It is recommended only for compatibility with existing projects.

```
# The following steps require Ubuntu to have internet access
sudo apt install -y curl
# Import the ROS repository signing key and add the ROS1 repository
sudo curl -sSL
https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc -o
/usr/share/keyrings/ros-archive-keyring.asc
echo "deb [signed-by=/usr/share/keyrings/ros-archive-keyring.asc]
http://packages.ros.org/ros/ubuntu focal main" | sudo tee
/etc/apt/sources.list.d/ros1.list
# Install ROS Noetic and the sample dependencies
sudo apt update
sudo apt install -y ros-noetic-desktop-full python3-rosdep
sudo apt install -y ros-noetic-cv-bridge ros-noetic-image-transport ros-
noetic-pcl-ros
sudo apt install -y libpcl-dev libopencv-dev
```

Add the ROS environment to the current user's configuration and initialize 'rosdep':

```
# Configure the ROS1 environment and apply it to subsequent terminals
echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
source /opt/ros/noetic/setup.bash
```

```
# Initialize rosdep the first time it is used; skip the first command if it
has already been initialized
sudo rosdep init
rosdep update
```

***Repeated initialization:** If `sudo rosdep init` reports that the configuration file already exists, run `rosdep update` directly.

5.2 Build and Start the Standard Camera Node

From the MRDVS directory, run:

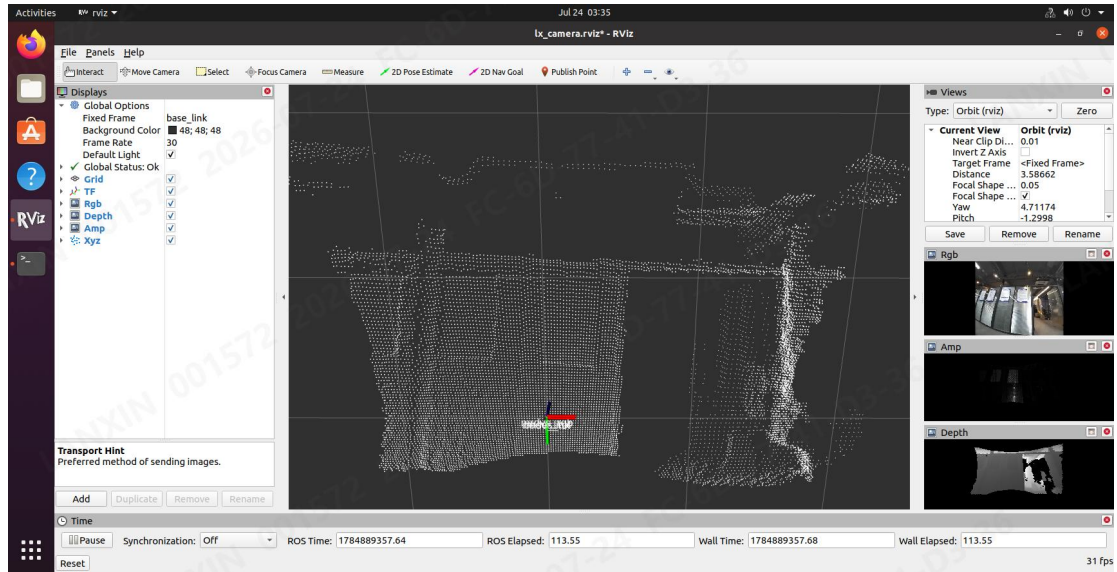
```
# Enter the ROS1 workspace and install missing dependencies
cd Sample/ROS/lx_camera_node_ws
rosdep install --from-paths src --ignore-src -r -y
# Build the workspace and source the runtime environment
catkin_make
source devel/setup.bash
# Start the standard camera node
roslaunch lx_camera_ros lx_camera_ros.launch
```

Open another terminal and run the following commands to verify the node, topics, and frame rates:

```
# Reload the ROS1 and workspace environments in the new terminal
source /opt/ros/noetic/setup.bash
cd Sample/ROS/lx_camera_node_ws
source devel/setup.bash
# Check that the node and topics are available
rostopic list
rostopic list
# Check the publication rates of the depth image, RGB image, and point
cloud
timeout 8s rostopic hz /lx_camera_node/LxCamera_Depth
timeout 8s rostopic hz /lx_camera_node/LxCamera_Rgb
timeout 8s rostopic hz /lx_camera_node/LxCamera_Cloud
# Read one camera frame-rate message
rostopic echo -n 1 /lx_camera_node/LxCamera_FrameRate
```

To open RViz at the same time, run:

```
roslaunch lx_camera_ros lx_camera_ros.launch enable_rviz:=true
```



5.3 Launch Files

File	Purpose	Requirements
lx_camera_ros.launch	Standard depth, RGB, point-cloud, and status node.	General entry point. Opens the device at index 0 by default.
lx_lidar_ros.launch	XYZIRT LiDAR point-cloud and IMU node.	Only for models that support LiDAR point clouds or an IMU.
mapping.launch	Mapping for the visual localization module.	Requires the actual IP address, external sensor topics, and extrinsic parameters.
localization.launch	Visual localization.	Requires the actual IP address, a valid map name, odometry, and extrinsic parameters.
sensor_sim.launch	Publishes simulated laser, odometry, and laser-pose data.	Used only for integration testing of the localization path; it does not replace real sensors.
obstacle.launch	Legacy obstacle-avoidance V1 configuration.	The parameter names use the legacy interface. Configure LX_INT_ALGORITHM_MODE=1 in the general launch file instead.
obstacleV2.launch	Legacy obstacle-avoidance V2 configuration.	The parameter names use the legacy interface. Configure LX_INT_ALGORITHM_MODE=4 in the general launch file instead.
pallet.launch	Legacy pallet-algorithm configuration.	The parameter names use the legacy interface. Configure LX_INT_ALGORITHM_MODE=2 in the general launch file instead.

***Starting an algorithm:** The standard node reads only the current parameters whose names begin with 'LX_'. 'LX_INT_ALGORITHM_MODE' is configured in the ROS1 'lx_camera_ros.launch'; when using a built-in algorithm, change its value directly. The corresponding ROS2 parameter is commented out by default; uncomment it and set the

appropriate value. Legacy launch files such as ‘obstacle’, ‘obstacleV2’, and ‘pallet’ still use legacy parameter names and are not recommended as entry points for the current version.

***Helper scripts:** ‘rate.sh’, ‘obstacle.sh’, and ‘pallet.sh’ subscribe to the corresponding ROS1 topics and must be run from the ‘lx_camera_node_ws’ directory. ‘build.sh’ deletes the existing ‘build’ and ‘devel’ directories before rebuilding. For routine incremental builds, use ‘catkin_make’ directly.

5.4 Standard Node Parameters

Parameter	Meaning	Description
ip	Device selection	0 or an empty string opens the device at index 0. A complete IPv4 address opens the device by IP address.
log_level / log_path	SDK logs	The default log level is 1. The log directory must be writable by the current user.
is_depth	Publish the depth image	1 enables it; 0 disables it.
is_xyz	Point-cloud type	0 disables point clouds; 1 publishes an XYZ point cloud; 2 publishes a LiDAR point cloud with intensity and time information.
LX_BOOL_ENABLE_3D_AMP_STREAM	3D amplitude stream	1 enables it; 0 disables it.
LX_BOOL_ENABLE_2D_STREAM	RGB stream	1 enables it; 0 disables it.
LX_BOOL_ENABLE_IMU	IMU	1 enables it. Unsupported models return ‘LX_W_NOT_SUPPORT’.
LX_INT_XYZ_UNIT	Point-cloud unit	0 means millimeters; 1 means meters. RViz normally uses meters.
LX_INT_XYZ_COORDINATE	Point-cloud coordinate frame	0 is the camera coordinate frame; 1 is the robot coordinate frame.
x/y/z/yaw/roll/pitch	Mounting extrinsic parameters	Translation is in meters; rotation is in degrees.

5.5 Topics and Services

Topic	Type	Content
LxCamera_Rgb	sensor_msgs/Image	RGB image.
LxCamera_Depth	sensor_msgs/Image	Depth image.
LxCamera_Amp	sensor_msgs/Image	Amplitude image.

Topic	Type	Content
LxCamera_RgbInfo / LxCamera_TofInfo	sensor_msgs/CameraInfo	RGB and ToF camera calibration information.
LxCamera_Cloud	sensor_msgs/PointCloud2	XYZ or RGB point cloud.
LxCamera_LidarCloud	sensor_msgs/PointCloud2	XYZIRT LiDAR point cloud.
LxCamera_Imu	sensor_msgs/Imu	IMU data.
LxCamera_FrameRate	lx_camera_ros/FrameRate	Depth, amplitude, and RGB frame rates and temperature.
LxCamera_Pallet	lx_camera_ros/Pallet	Pallet-algorithm results.
LxCamera_Obstacle	lx_camera_ros/Obstacle	Obstacle-avoidance algorithm results.

The standard node also provides the ‘LxCamera_LxCmd’, ‘LxCamera_LxInt’, ‘LxCamera_LxBool’, ‘LxCamera_LxFloat’, and ‘LxCamera_LxString’ services for accessing command, integer, Boolean, floating-point, and string features, respectively. Their fully qualified names are under the /lx_camera_node namespace.

6 ROS2 Samples

6.1 Install ROS2 Humble and Its Dependencies

The ROS2 samples target Ubuntu 22.04 and Humble. On a clean system, install them as follows:

```
# The following steps require Ubuntu to have internet access
sudo apt update
sudo apt install -y software-properties-common curl
sudo add-apt-repository -y universe
# Import the ROS2 repository signing key and add the Humble repository
sudo curl -sSL
https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
/usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/usr/share/keyrings/ros-archive-keyring.gpg]
http://packages.ros.org/ros2/ubuntu jammy main" | sudo tee
/etc/apt/sources.list.d/ros2.list
# Install ROS2 Humble, the build tools, and the sample dependencies
sudo apt update
sudo apt install -y ros-humble-desktop python3-colcon-common-extensions
python3-rosdep
sudo apt install -y ros-humble-cv-bridge ros-humble-image-transport
```

```
sudo apt install -y ros-humble-pcl-conversions ros-humble-tf2-ros libpcl-
dev libopencv-dev
# Configure the ROS2 environment and apply it to subsequent terminals
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source /opt/ros/humble/setup.bash
# Initialize rosdep the first time it is used; skip the first command if it
has already been initialized
sudo rosdep init
rosdep update
```

6.2 Build, Start, and Verify

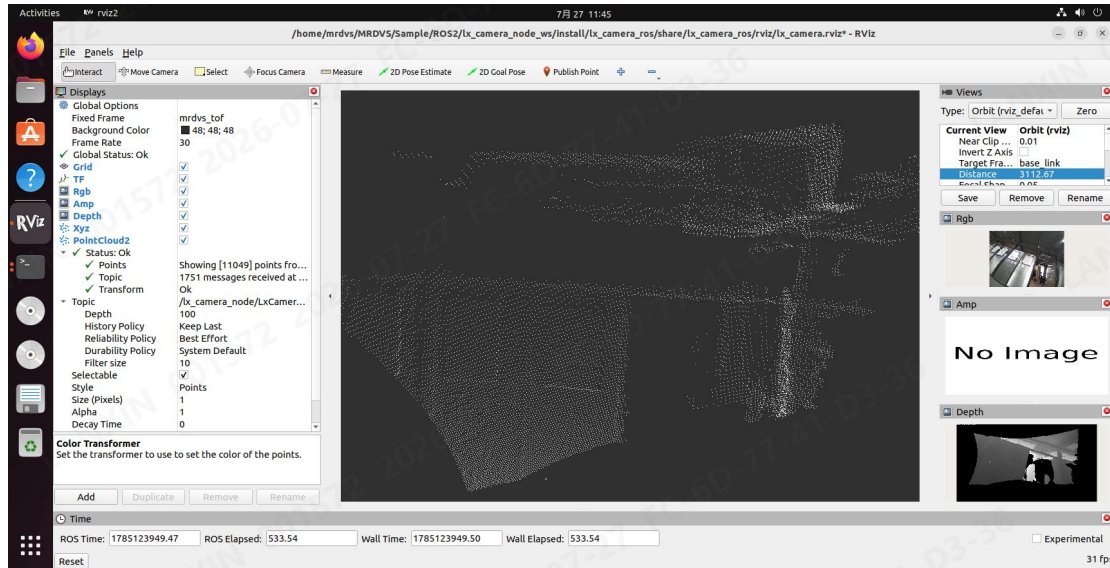
From the MRDVS directory, run:

```
# Enter the ROS2 workspace and install missing dependencies
cd Sample/ROS2/lx_camera_node_ws
rosdep install --from-paths src --ignore-src -r -y
# Build the workspace and source the runtime environment
colcon build
source install/setup.bash
# Start the standard camera node
ros2 launch lx_camera_ros lx_camera_ros.launch.py
# The standard camera node disables the amplitude stream by default. Enable
it with the enable_amp parameter when an amplitude image is required
ros2 launch lx_camera_ros lx_camera_ros.launch.py enable_amp:=1
# Open RViz2 at the same time
ros2 launch lx_camera_ros lx_camera_ros.launch.py enable_rviz:=true
```

Open another terminal for verification:

```
# Reload the ROS2 and workspace environments in the new terminal
source /opt/ros/humble/setup.bash
cd Sample/ROS2/lx_camera_node_ws
source install/setup.bash
# Check that the node and topics are available
ros2 node list
ros2 topic list
# Check the depth-image publication rate; press Ctrl+C after observing it
ros2 topic hz /lx_camera_node/LxCamera_Depth
# Read one camera frame-rate message
ros2 topic echo --once /lx_camera_node/LxCamera_FrameRate
```

***Modifying a launch file:** Modify the source file in `src/lx_camera_ros/launch`, then run `colcon build` again and source `install/setup.bash`. Do not modify only the generated copy in the `install` directory.



6.3 CycloneDDS

If the node runs but the frame rate at the subscriber is significantly low, install CycloneDDS and select this RMW implementation in the terminal used to start the node:

```
# Install and enable the CycloneDDS communication implementation
sudo apt install -y ros-humble-rmw-cyclonedds-cpp
echo "export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp" >> ~/.bashrc
source ~/.bashrc
# Restart the node with the new DDS configuration
ros2 launch lx_camera_ros lx_camera_ros.launch.py
```

ROS2 provides `'lx_camera_ros.launch.py'`, `'lx_lidar_ros.launch.py'`, `'mapping.launch.py'`, `'localization.launch.py'`, `'sensor_sim.launch.py'`, `'obstacle.launch.py'`, `'obstacleV2.launch.py'`, and `'pallet.launch.py'`, with functions corresponding to their ROS1 counterparts. When `'lx_camera_ros.launch.py'` is used, the fully qualified name of the standard node is under the `'/lx_camera_node'` namespace, and the localization node is under the `'/lx_localization_node'` namespace.

***Helper scripts:** `'rate.sh'`, `'obstacle.sh'`, and `'pallet.sh'` in the ROS2 directory use topic names without namespaces. Use the fully qualified `'/lx_camera_node/LxCamera_*` names provided in this chapter instead. `'build.sh'` deletes the `'build'`, `'install'`, and `'log'` directories. For routine incremental builds, use `'colcon build'` directly.

7 Visual Localization, Mapping, and Relocalization

Visual localization is available only on cameras with the corresponding algorithm deployed. Before use, confirm the camera algorithm version, map format, external sensor data, and mounting extrinsic parameters. Both ROS1 and ROS2 provide the ‘lx_localization_node’, ‘mapping’, ‘localization’, and ‘sensor_sim’ samples.

7.1 Inputs, Outputs, and Key Configuration

Item	Content
Mapping input	Absolute odometry (‘nav_msgs/Odometry’); depending on the application, a LaserScan and laser pose (‘PoseStamped’).
Localization input	Absolute odometry; an initial pose during relocalization.
Localization output	‘LxCamera_LocationResult’, of type ‘geometry_msgs/PoseStamped’.
Mapping mode	‘mapping_mode=true, localization_mode=false’.
Localization mode	‘mapping_mode=false’, ‘localization_mode=true’, with a valid ‘map_name’ that already exists in the camera.
Extrinsic parameters	‘camera_extrinsic_param’ format: [x,y,z,yaw,pitch,roll]; ‘laser_extrinsic_param’ format: [x,y,yaw].
Simulated topics	‘/sim/scan’, ‘/sim/odom’, and ‘/sim/scan_pose’; used only for integration testing of the data path.

7.2 Integration Sequence

1. In ‘mapping.launch’ or ‘localization.launch’, set the actual camera IP address, topic names, map name, and extrinsic parameters. The sample IP address and map name in the localization file cannot be used directly at another site.
2. Start ‘sensor_sim’ and confirm that the simulated laser, odometry, and laser-pose topics are published continuously.
3. Start the ‘mapping’ node. Status 20 indicates normal mapping. Before switching to real sensors, confirm that the topic types, timestamps, and coordinate frames match.
4. After mapping is complete, disable mapping before exporting the raw data. Generate and deploy the offline map according to the project delivery process.
5. After importing a valid map, start the ‘localization’ node. When sending a relocalization pose, do not set ‘x’, ‘y’, and ‘yaw’ all to 0; otherwise, the request may be treated as an accidental command.

```
# ROS1: Start the sensor simulation node in terminal 1
```

```

roslaunch lx_camera_ros sensor_sim.launch
# ROS1: Start the mapping node in terminal 2
roslaunch lx_camera_ros mapping.launch
# ROS2: Start the sensor simulation node in terminal 1
ros2 launch lx_camera_ros sensor_sim.launch.py
# ROS2: Start the mapping node in terminal 2
ros2 launch lx_camera_ros mapping.launch.py

```

***Detailed procedure:** For map import/export, mapping-data delivery, relocalization messages, and all status codes, see ‘README_Localization(ROS1).pdf’ or ‘README_Localization(ROS2).pdf’ in the software package. Map filename extensions may vary with the camera algorithm version; follow the requirements of the actual device.

8 Troubleshooting

Symptom	Cause and Resolution
No devices are discovered	Confirm that the host and camera are on the same subnet, the network adapter is connected, the route is correct, and the firewall is not blocking the SDK ports.
Python reports "file libLxCameraApi.so does not exist"	The Python sample still uses a relative shared-library path. Change the path to /opt/MRDVS/lib/libLxCameraApi.so as described in Section 4.2, and run single_camera.py from the current directory.
A shared library or one of its dependencies fails to load	Run <code>sudo ldconfig</code> to refresh the cache, then use <code>ldconfig -p</code> and <code>ldd /opt/MRDVS/lib/libLxCameraApi.so</code> to check for missing dependencies.
LX_E_CTRL_PERMISS_ERROR	The camera is in use by another process. Close other demos, ROS nodes, and host applications, wait for the camera to release control after the control heartbeat times out, and try again.
LX_W_NOT_SUPPORT	The current camera does not support the configured IMU, algorithm, data stream, or parameter. Disable the corresponding configuration according to the model's capabilities.
Only some images or point clouds are available	Check the camera stream switches, <code>is_depth</code> , <code>is_xyz</code> , <code>LX_BOOL_ENABLE_2D_STREAM</code> , and <code>LX_BOOL_ENABLE_3D_STREAM</code> .
The image window does not open	The current terminal has no graphical environment, or <code>DISPLAY</code> is unavailable. Use display-free data processing, or run the sample on a local graphical desktop.
The ROS node exits shortly after startup	Common causes include an IP address mismatch, control access held by another process, an unwritable log directory, or an unsupported device-feature configuration. Check the first error in the startup terminal.
"new node registered with same name" appears	A ROS node with the same name is already running. Stop the existing node before running <code>roslaunch</code> again.

Low ROS2 topic frame rate	Switch to CycloneDDS, and check the subscriber QoS, CPU load, and network bandwidth.
CMake reports a PCL directory linking warning	The current project treats some PCL include directories as link items. If the node is generated successfully and runs normally, record the warning. If the build fails, check the PCL and Eigen installations.
The camera cannot be restarted immediately after Ctrl+C	The sample did not complete the normal shutdown sequence. Confirm that the process has ended and wait for the device to be released. Production code should implement signal handling and resource cleanup.

After basic verification, use 'single_camera2' or the standard ROS node as the minimum runnable baseline, then enable RGB, amplitude, IMU, LiDAR point clouds, and built-in algorithms one at a time. Adding only one configuration item at a time makes it easier to distinguish device capabilities, network issues, and software configuration issues.